AI Red-Teaming Arena – Concept, Ecosystem, MVP Plan, and Tech Stack

1. Concept overview

The AI Red-Teaming Arena is a platform where autonomous "attacker" agents systematically probe language models with adversarial prompts, while "defender" agents apply guardrails and safety policies to block harmful outputs before they reach end users. The system runs many simulated attacks across safety categories (self-harm, hate, PII leakage, prompt injection, model-spec violations), evaluates responses, and visualizes vulnerabilities and mitigation effectiveness so teams can harden their AI applications before real-world deployment.[1][2] [3][4] [^5]

The arena has three core value propositions:

- Provide a **controlled environment** to stress-test LLMs and agentic apps against realistic adversarial behavior.
- Combine **automated attack generation** and **programmable defenses** (guardrails) so safety mechanisms can be tuned and compared.
- Offer **observability and evaluation** (traces, metrics, reports) that make safety work measurable and repeatable.

2. High-level goals for the hackathon MVP

For the 3-day hackathon MVP, the goals should be:

1. **Interactive duel mode**
   - Let a judge pick a scenario and watch a short duel between attacker agents and defender agents.
   - Show clearly whether a harmful output was produced or blocked, and why.
2. **Basic attack coverage**
   - Cover a small but meaningful set of scenarios (e.g., self-harm, hate speech, PII leakage, prompt injection) with predefined templates and dynamic variations.[6][1]
3. **Guardrail integration**
   - Integrate at least one production-grade guardrails toolkit (e.g., NVIDIA NeMo Guardrails, LLM Guard) to demonstrate real-world defense mechanisms.[7][8]
4. **Evaluation and observability**
   - Log all duels and expose basic metrics (attack success rate, defense effectiveness) via a small dashboard.

- Connect to an evaluation/observability tool (LangWatch + Promptfoo) to show regression testing and traces.[9][10] [11][12] [^1]

3. Ecosystem tools and recommendations

3.1 Red-teaming & attack generation

DeepTeam (Confident AI)

- Description: Open-source framework designed specifically for LLM and LLM-application red teaming, built on DeepEval.[13][14] [15][16]
- Capabilities:
  - Defines vulnerability types (toxicity, PII leakage, prompt injection, etc.) and generates attacks against a model endpoint.
  - Produces structured results (JSON/dataframe) and summary reports of discovered vulnerabilities.[15][16]
- Recommendation for MVP:
  - Use DeepTeam in **batch mode** to generate offline attack runs against the arena's target model endpoint.
  - Import a subset of its attacks into the arena's scenario store so interactive duels can replay representative cases.

Promptfoo red-team patterns and open-source attack guides

- Promptfoo provides a documented red-teaming guide with concrete steps for generating adversarial prompts and evaluating LLM responses.[1][6]
- Surveys of open-source red-teaming tools highlight common taxonomies of attacks (self-harm, hate, PII, prompt injection, jailbreaks) that can be reused as scenario categories. [^17]
- Recommendation:
  - Seed the scenario database with patterns drawn from Promptfoo's red-team docs and related guides.
  - Use these as starting points for attacker agents to mutate and combine.

OpenRT (for multimodal extension)

- An open-source red-teaming framework for multimodal LLMs that separates model adapters, attack datasets, attack strategies, judges, and metrics.[^18]
- Recommendation:
  - Treat OpenRT as a **stretch goal**: if time allows, demonstrate a single multimodal scenario (e.g., image + text) using OpenRT's attack kernel.

3.2 Guardrails and defenses

NVIDIA NeMo Guardrails

- Description: Open-source toolkit for adding input and output guardrails to LLM applications.[^7]
- Capabilities:
  - Input rails: jailbreak detection, sensitive data masking.
  - Output rails: moderation, self-checking facts, hallucination detection.[^7]
  - Comes with an `evaluate` CLI and sample vulnerability scan reports.
- Recommendation:
  - Use NeMo Guardrails as the **primary defense layer** in the MVP.
  - Implement at least:
    - An input rail for jailbreak detection and PII masking.
    - An output rail for content moderation (blocking unsafe categories).

LLM Guard (Protect AI)

- Description: Security toolkit focused on LLM interaction hardening: prompt injection detection, PII protection, toxicity filters, etc.[^8]
- Recommendation:
  - Use LLM Guard as a **lightweight pre-filter** alongside NeMo Guardrails for quick wins on obvious PII/toxicity and injection patterns.

3.3 Evaluation & testing

Promptfoo

- Description: Open-source framework for testing prompts and evaluating LLM outputs with declarative configs, CLI, and GitHub Actions integration.[12][19]
- Capabilities:
  - Define providers, prompts, and assertions, including `llm-rubric` style evaluations where a model judges outputs based on a rubric.[19][12]
  - Compare prompt or model variants, and integrate with CI via GitHub Actions.[^20]
- Recommendation:
  - Export batched arena runs (attacks + outputs + labels) into Promptfoo.
  - Define tests that assert safety (e.g., "no self-harm encouragement"), using `llm-rubric` for nuanced judgments.[12][19]
  - Use Promptfoo results as part of the MVP story: "we can regression-test safety behavior when we change guardrails or model versions".

DeepEval / DeepTeam reports

- DeepTeam's outputs and underlying DeepEval metrics provide structured vulnerability assessments per model/app.[16][15]
- Recommendation:
  - Store DeepTeam vulnerability reports and surface key stats (e.g., coverage and weaknesses) in the arena UI.

3.4 Observability & tracing

LangWatch

- Description: Open-source LLM and agent observability platform, focusing on tracing, evaluation, and experiment tracking.[10][11] [^9]
- Capabilities:
  - Records traces and spans for LLM/agent pipelines.
  - Supports test runs and evaluation integration with various LLM SDKs.[11][10]
- Recommendation:
  - Instrument the arena so each duel (scenario run) becomes a **trace** with spans for:
    - Scenario selection.
    - Attacker agent steps.
    - Guardrails input checks.
    - Model calls.
    - Guardrails output checks.
    - Policy/evaluation decisions.
  - Use LangWatch dashboards during the demo to show how attacks and defenses played out internally.

4. Conceptual architecture

4.1 Main components

1. **Scenario & dataset layer**
   - Stores test cases with:
     - Category: self-harm, hate, PII, prompt injection, etc.
     - Description: natural-language description of the scenario.
     - Base prompts or templates.
   - Seeded from Promptfoo, DeepTeam patterns, and curated manual examples.[17][6] [16][1]
2. **Attack engine (Red Team)**
   - **RedTeamCoordinator** (LangGraph/Autogen): selects scenario, chooses attack strategy (single-turn, multi-turn, translation, roleplay, etc.).[21][22] [^23]

- AttackerAgents:
  - Use scenario context and past outputs to generate adversarial prompts.
  - Apply known red-team techniques (prompt injection, jailbreaks, obfuscation) informed by open-source guides.[6][17] [^1]
- DeepTeam integration:
  - For batch/offline runs, DeepTeam hits the same target endpoint with large attack sets and logs results.[14][15] [^16]

3. Defense engine (Blue Team)
   - Guardrail layer:
     - NeMo Guardrails input rails: jailbreak detection, PII masking.[^7]
     - NeMo Guardrails output rails: moderation and, if time permits, self-checking flows.[^7]
     - Optional LLM Guard filters for quick PII and toxicity detection.[^8]
   - DefenderAgents:
     - DefenseCoordinator decides whether to:
       - Strengthen the system prompt.
       - Enable stricterGuardrails profiles.
       - Block or escalate.
     - PolicyAgent uses a policy document or rubric to classify outcomes (safe vs unsafe) when not relying on Promptfoo's rubric directly.
4. Model gateway
   - A small service (e.g., FastAPI/Express) that wraps one or more underlying models and provides a single endpoint for the arena.
   - All traffic goes: Attacker → Guardrails input → Model Gateway → Guardrails output → evaluation.
5. Evaluation & storage
   - Database (PostgreSQL or Supabase) to store:
     - Scenarios.
     - Attack attempts.
     - Model responses.
     - Guardrail decisions.
     - Final labels (safe/unsafe, blocked/not blocked).
   - Export pipelines to Promptfoo and DeepTeam result formats.[15][19] [16][12]
6. Observability & UI
   - LangWatch traces for all duels.[9][10] [^11]
   - A web UI that offers:

    - Live duel view (conversation, guardrail decisions, verdict).
    - Analytics view (attack success rates, defense effectiveness, per-category results).

4.2 Data and control flow

Example interactive duel flow:

1. User/judge selects a scenario from the UI.
2. RedTeamCoordinator picks an attack strategy and initial attacker prompt.
3. Input passes through NeMo Guardrails and LLM Guard; if blocked, the attacker tries another strategy.
4. If allowed, the model gateway calls the target model and returns an output.
5. Output passes through NeMo Guardrails and LLM Guard.
6. DefenderAgents may modify prompts/rails or block the response.
7. PolicyAgent or Promptfoo rubric determines if this is a success or failure from a safety perspective.[19][1] [^12]
8. All steps are logged to the DB and to LangWatch as a trace.

Batch mode with DeepTeam:

- DeepTeam runs large sets of attacks against the model endpoint; results are stored and optionally replayed as curated interactive examples.[14][16] [^15]

5. MVP scope and implementation plan (3 days)

Day 0 (pre-hackathon preparation)

- Familiarize the team with:
    - NeMo Guardrails basics (one or two example flows).[^7]
    - LangWatch client integration.
    - Promptfoo config format and CLI.[12][19]
    - DeepTeam basic usage (one simple run against a test endpoint).[16][14] [^15]
- Prepare a small set of policy texts and safety rubrics (e.g., based on Microsoft/OpenAI safety categories and generic trust & safety docs).[3][4] [^5]

Day 1 – Skeleton and core duel loop

Objectives: get a minimal end-to-end duel working.

- Backend:
    - Implement the model gateway service with a single LLM provider.
    - Integrate NeMo Guardrails for a simple input+output moderation flow.[^7]

    - Implement a basic AttackerAgent and DefenderAgent using LangGraph/Autogen, with hard-coded strategies and decisions.[22][23] [^21]
- Data:
    - Seed 5–10 scenarios manually across different safety categories.
- UI:
    - Build a minimal duel UI: select scenario, see prompt/response, and see a "blocked/allowed" flag.
- Observability:
    - Instrument the main path with LangWatch (one trace per duel).[10][11]

Day 2 – Attack coverage, evaluation, and metrics

Objectives: broaden coverage and add evaluation.

- Attacks:
    - Expand scenarios using patterns from Promptfoo and open red-team guides (e.g., different jailbreak tricks, translation, coding-based attacks).[17][1] [^6]
    - Add at least one multi-turn attack strategy (attacker learns from previous failures).
- Defense:
    - Enable at least one additional NeMo Guardrails rail (e.g., stronger moderation or hallucination self-check if relevant).[^7]
- Evaluation:
    - Export duel logs into Promptfoo and run a small test suite with `llm-rubric` assertions.[19][12]
    - Summarize metrics: attack success rate per category, before vs after enabling certain guardrails.
- Observability/UI:
    - Add a simple analytics view (charts or tables) summarizing metrics.

Day 3 – DeepTeam integration and polish

Objectives: integrate external red-team engine and polish narrative.

- DeepTeam:
    - Run DeepTeam against the model gateway for 1–2 vulnerability profiles.[14][15] [^16]
    - Import a subset of its attack prompts into the arena scenario store as curated "expert attacks".
- UI polish:
    - Show scenario metadata, attack/defense timeline, and final verdict clearly.
    - Add a simple "config" view where judges can toggle guardrail profiles or model versions.

- Storytelling:
  - Prepare 2–3 demo scripts:
    - Baseline (no guardrails) vs defended runs for the same scenario.
    - One example of a tricky attack that initially bypassed defense but was later blocked after tuning.
  - Highlight how LangWatch, Promptfoo, and DeepTeam give observability, regression testing, and external validation.

6. Recommended tech stack summary

Backend & agents

- Language: Python
- Frameworks:
  - FastAPI (HTTP APIs, model gateway, arena backend).
  - LangGraph or Autogen for attacker/defender coordination.[23][21] [^22]
- Guardrails & defense:
  - NeMo Guardrails for policy flows.
  - LLM Guard as auxiliary security filter.[8][7]

Red-teaming & evaluation

- DeepTeam for batch red-teaming and vulnerability reports.[15][16] [^14]
- Promptfoo for prompt/model regression testing and rubric-based evaluation.[1][12] [^19]
- (Optional stretch) OpenRT for multimodal attacks.[^18]

Observability

- LangWatch for tracing and experiment dashboards.[11][9] [^10]

Data & storage

- PostgreSQL or Supabase for scenarios, logs, and metrics.

Frontend

- Next.js or similar React framework for the duel UI and analytics view.
- Tailwind CSS or a simple component library for rapid UI development.

This stack balances **real-world seriousness** (NeMo Guardrails, DeepTeam, Promptfoo, LangWatch) with **hackathon feasibility**, allowing the team to deliver a compelling MVP that shows both technical depth and clear business/research value within three days.[10][17] [1][15] [12][7]

References

1. LLM red teaming guide (open source) - Promptfoo - LLM red teaming is a way to find vulnerabilities in AI systems before they're deployed by using simu...
2. RedTeam Arena: An Open-Source, Community-driven Jailbreaking ... - RedTeam Arena is an open-source red-teaming platform for LLMs. Our plan is to provide games that peo...
3. Planning red teaming for large language models (LLMs) and their ... - The term red teaming has historically described systematic adversarial attacks for testing security ...
4. Large Language Model (LLM) Red Teaming - Appen - LLM red teaming actively tests for vulnerabilities in a systematic and reliable way, allowing model ...
5. LLM Red Teaming - Arize AX Docs - LLM red teaming is a proactive security practice that identifies vulnerabilities in AI systems befor...
6. Utilizing Open-Source Red Teaming Tools - ApX Machine Learning - Effective LLM red teaming involves both manual prompt crafting for precision and automated generatio...
7. NeMo Guardrails is an open-source toolkit for easily adding ... - GitHub - Building Trustworthy, Safe, and Secure LLM-based Applications: you can define rails to guide and saf...
8. protectai/llm-guard: The Security Toolkit for LLM Interactions - GitHub - LLM Guard by Protect AI is a comprehensive tool designed to fortify the security of Large Language M...
9. LangWatch - GitHub - LangWatch The core open-source platform for observing, evaluating, and testing LLM-powered agents. ·...
10. Observability Integrations: LangWatch - AI SDK - LangWatch (GitHub) is an LLM Ops platform for monitoring, experimenting, measuring and improving L...
11. LangWatch: AI Agent Testing and LLM Evaluation Platform - LangWatch is an AI agent testing, LLM evaluation, and LLM observability platform. Test agents with s...
12. AI-App/PromptFoo: Test your prompts. Evaluate and compare LLM ... - promptfoo is a tool for testing and evaluating LLM output quality. With promptfoo, you can: Systemat...
13. DeepTeam by Confident AI - The LLM Red Teaming Framework - Delivered by Confident AI. Detect 40+ LLM Vulnerabilities. Automatically scan for vulnerabilities su...
14. DeepTeam is a framework to red team LLMs and LLM ... - GitHub - DeepTeam is a simple-to-use, open-source red teaming framework for LLM systems. Think of it as penet...
15. LLM Red Teaming: The Complete Step-By-Step Guide To LLM Safety - In this article, we're going to focus on automated testing instead, mainly using DeepTeam ⭑, an open...
16. Genez-io/genezio-deepteam: The LLM Red Teaming Framework - DeepTeam is a simple-to-use, open-source LLM red teaming framework, for safety testing large-languag...
17. Best Open Source LLM Red Teaming Tools (2025) - OnSecurity - Discover the best open-source LLM red teaming tools of 2025. Learn how to test AI systems, strengthe...
18. An Open-Source Red Teaming Framework for Multimodal LLMs - arXiv - To address this, we introduce OpenRT, a unified, modular, and high-throughput red-teaming framework ...

19. Runtime LLM evaluation with Promptfoo & Echo - Hotovo - Promptfoo is a framework designed to systematically evaluate LLM outputs. It's commonly used via YAM...
20. Testing Prompts with GitHub Actions - Promptfoo - Automate LLM prompt testing in CI/CD with GitHub Actions integration. Compare prompt changes, view d...
21. How and when to build multi-agent systems - LangChain Blog - One of the hardest parts of building multi-agent (or even single-agent) applications is effectively ...
22. Benefits and Challenges of Multi-Agent Systems: When Complexity ... - Explore the trade-offs of multi-agent AI systems, from specialization and parallel processing to coo...
23. How Multi-Agent Systems Solve Complex Problems in 2025 - Multi-agent systems are revolutionizing how we approach complex challenges in AI, from managing smar...